

Delicut

TL;DR

Delicut is a UAE-based meal plan service with 15,000+ customers.

I worked on the web subscription flow and mobile app, focusing on onboarding, meal plan management, delivery states, renewal, and implementation QA.

A key part of the work was defining how the product should behave across different subscription states — before a plan starts, while it is active, when deliveries are skipped or locked, and after the plan expires.

Role

Product Designer

Web + Mobile

Worked on

Onboarding, subscription UX, delivery management, IA, mobile flows, design QA

Team

Design, product, client stakeholders, engineering

The problem

The product covered the main subscription flow, but many real user states were unclear or missing.

For example:

- plan has not started yet
- today's meals are locked
- future meals can still be edited
- a delivery is skipped
- a user tries to resume after cut-off
- a plan is about to expire
- a plan has already expired

The challenge was to make these states clear without making the app feel complicated.

What I found

As we mapped the mobile experience, the number of states grew quickly.

We documented **27 home and delivery scenarios**, including skipped days, future meal assignment, cut-off rules, mixed active and paused days, expiry and renewal.

This showed that the problem was not just screen design. The product needed clearer rules for how each state should behave.

What I focused on

1. Simplifying the web subscription flow

We reworked the web journey into a more familiar sequence:

Select plan → Add delivery details → Review → Pay

We also reduced early sign-up friction by collecting user details as part of the checkout flow instead of stopping users with a separate account step.

2. Defining subscription states in the mobile app

I helped structure the app around a few key variables:

Plan status

Not started / Active / Expiring / Expired

Day

Past / Today / Future

Delivery state

Active / Skipped / Locked

Action

View / Edit / Skip / Resume / Rate / Renew

This made it easier to define what the user should see and what actions should be available in each case.

3. Making time-based rules easier to understand

The same delivery could behave differently depending on when the user viewed it.

Today

View only if the cut-off has passed

Tomorrow

Change meal, skip, or change address before cut-off

Yesterday

View delivered meal and rate it

This helped make product rules visible through the interface instead of relying on users to remember them.

4. Simplifying skip and resume

Skipping had several states: one day, multiple days, full pause, partial resume, or trying to resume after cut-off.

We moved toward a day-based calendar model so users could see which delivery dates were active and which were skipped.

5. Planning for expiry and renewal

We created separate states for:

Plan expiring soon

and

Plan expired

This gave users a clear path to renew instead of leaving them in an inactive or unclear state.

Working through changing requirements

The project changed often during design.

Ideas were introduced live in reviews, requirements shifted, and some screens changed while development was already in progress.

To keep the work consistent, we started documenting decisions and checking changes against the wider flow instead of updating screens in isolation.

This helped reduce repeated design changes and made discussions more focused.

Design QA

I also stayed involved during implementation.

The QA process covered visual issues, interaction problems, missing states, content errors and flow logic across iOS and Android.

This helped catch differences between the intended experience and the built product before release.

Outcome

The project resulted in a clearer structure for both the web subscription journey and the mobile app.

The main improvement was moving from isolated screens to defined product states that could handle different points in the subscription lifecycle.

That gave the team a more consistent base for onboarding, delivery management, expiry, renewal and future features.

Reflection

This project improved how I think about product logic.

The biggest design task was often not creating a new screen, but deciding:

What state is the user in?

What should they see?

What can they do right now?

That approach made the product easier to design, review and build.